ML-CGRA: An Integrated Compilation Framework to Enable Efficient Machine Learning Acceleration on CGRAs

Yixuan Luo*§, Cheng Tan*¶, Nicolas Bohm Agostini†, Ang Li†, Antonino Tumeo†, Nirav Dave¶, Tong Geng§
§University of Rochester, †Pacific Northwest National Laboratory, ¶Microsoft
§{yluo37, tong.geng}@rochester.edu, ¶{cheng.tan, nirav.dave}@microsoft.com,
†{nicolas.agostini, ang.li, antonino.tumeo}@pnnl.gov

2023 60th ACM/IEEE Design Automation Conference (DAC) | 979-8-3503-2348-1/23/$31.00 ©2023 IEEE | DOI: 10.1109/DAC56929.2023.10247873

***Abstract*—Coarse-Grained Reconfigurable Arrays (CGRAs) can achieve higher energy-efficiency than general-purpose processors and accelerators or fine-grained reconfigurable devices, while maintaining adaptability to different computational patterns. CGRAs have shown some success as a platform to accelerate machine learning (ML) thanks to their flexibility, which allows them to support new models not considered by fixed accelerators. However, current solutions for CGRAs employ low level instruction-based compiler approaches and lack specialized compilation infrastructures from high-level ML frameworks that could leverage semantic information from the models, limiting the ability to efficiently map them on the reconfigurable substrate. This paper proposes ML-CGRA, an integrated compilation framework based on the MLIR infrastructure that enables efficient ML acceleration on CGRAs. ML-CGRA provides an end-to-end solution for mapping ML models on CGRAs that outperforms conventional approaches by 3.15× and 6.02 × on 4×4 and 8×8 CGRAs, respectively. The framework is open-source and available from https://github.com/tancheng/mlir-cgra.**

I. Introduction

In the last decade, a plethora of machine learning (ML) applications have gained prominence across a number of fields to find fast solutions for challenging problems. As the scope of these problems increases in generality and improved results, the scale of both the underlying data and model complexity grows exponentially, placing increased pressure on implementations to evaluate these ML models quickly *and* efficiently, leading to the creation of many hardware-acceleration platforms. Due to the sheer scale of modern deployments and the desire to constantly improve model performance, being able to retarget a platform to support new algorithms in a short time is a crucial feature. Simultaneously total power usage provides a counterbalancing concern guiding hardware designs to specialized, more static hardware. Coarse-Grained Reconfigurable Arrays (CGRAs) [9], [2], [13], [10], [4], [12], [29] provide an interesting compromise of these features, being more flexible than specialized application-specific integrated circuit (ASIC) solutions but outperforming general purpose solutions (such as central processing units and graphic processing units - CPU/GPU) in terms of perf/watt.

While CGRAs offer the potential to achieve better power- and energy efficiency than alternatives, the standard CGRA compilation frameworks focus on inner-most loop kernel acceleration through dataflow graph (DFG) mapping [21], [18] through low level instruction-based compilation approaches. This provides generality, but leaves out the potential for a slew of coarser-grain considerations that when properly leveraged, could significantly improve end-to-end performance. For example, maintaining semantic information on ML operators and layers connections simplifies the mapping on predefined parametric templates. As a result, in practice, a user wishing to generate a high-performance implementation with conventional compilation flows must work around the compiler's decisions and manually refine the input model, inlining specialized kernels, fusing loops, and tuning parameters by hand. While doable, this approach dramatically reduces the attractiveness of the CGRAs of a substrate [7]. Alternatively, it has been shown that for some applications it is possible to implement design space exploration approaches that also consider compiler optimizations to obtain high-performance implementations [13], [12], [24], [20]. While these solutions allows to find better design points, they still operate at the instruction level and inherit a level of brittleness as the CGRA user cannot easily modify the generated code and needs to repeat the entire design space exploration process.

The goal of this work is to get the best of both CGRA compilation approaches and get the full flexibility of generic compilation passes while also exploiting higher-level structures in the code to improve overall performance. To achieve this, we propose ML-CGRA, an open-source integrated end-to-end compilation framework that enables efficient and flexible ML acceleration on CGRAs. The concrete contributions are the following:

- We implement a `cgra-dialect`: a generic computational dialect with first-order representational support for key ML acceleration operations.
- We provide a set of key optimization and lowering passes to generate performant implementations.
- We introduce ML-CGRA, our MLIR-based open-source compilation framework which is compatible with various ML front-ends (e.g., ONNX, PyTorch, TensorFlow) and is easily extendable.
- We implement a CGRA runtime library to facilitate the cycle-level and register-transfer-level (RTL) simulation. This allows users to evaluate at a low-level performance concerns for their models.

II. Related Work

Compared with GPU, FPGA, and ASIC, as one of the most computationally efficient and freely customizable architectures, CGRA has been considered a promising solution for accelerating massive complex computing in a dataflow fashion. Recently, researchers have demonstrated that CGRAs with dedicated designs for ML workloads achieve superior acceleration and power efficiency when executing Convolution

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 09:42:47 UTC from IEEE Xplore. Restrictions apply.

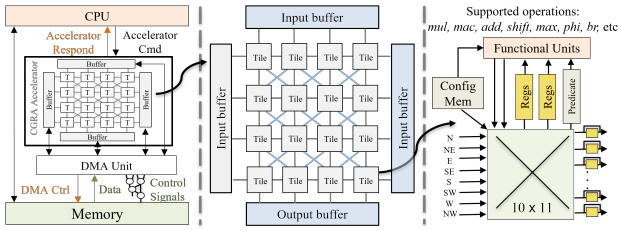
Neural Network (CNN) or Graph Neural Network (GNN) models [21], [3], [13], [10], [4], [12], [29].

To accelerate CNN computation, [21] proposes a novel CGRA-based approach that can effectively reduce memory bandwidth pressure by using distributed scratchpad memory blocks to achieve efficient temporal blocking. To further leverage the potential of CGRA, [3] demonstrates that CGRA can be specially designed to reuse the input data during multiple computation periods with multithreading at the processing element level, which significantly reduces the data communication overhead. With the emergence of novel ML algorithms, it is vital to adapt the CGRA to different ML tasks. Recent works [13], [10], [4], [12], [29] focus on different ML algorithms and structures and configure/customize the CGRA architectures to obtain optimal acceleration performance. However, each work provides custom configurations for either a subset of observed ML operations or, sometimes, individual operations, and there is no framework to automate this procedure across different ML models.

End-to-end frameworks were proposed to deploy the latest ML algorithms on different hardware platforms and effectively accelerate them for better performance. TVM [5] is a ML compiler stack that automatically optimizes computation at the computational graph and operator level for a diverse set of hardware platforms, such as FPGA, CPU, and GPU. Incorporating hardware information into the optimization pipeline, HASCO[25] presents a novel approach for hardware and software co-design algorithms (i.e., Bayesian optimization, heuristic, and Q-learning algorithms) to obtain better performance when compared with isolatedly developing hardware or software. Furthermore, to fairly evaluate the existing DNN accelerators and explore the future design, Timeloop [15] proposes a framework that provides an automated model to evaluate the DNN accelerators' performance among complex workloads. Extending MLIR [11], Scale-HLS [26] implemented high-level analysis and transformations to generate annotated C/C++ that guides its high-level synthesis (HLS) backend, Vivado HLS. Their flow results in custom FPGA designs for individual algorithms described with MLIR specifications. Furthermore, SODA-OPT [1] proposed an MLIR abstraction and compiler transformations to perform automated host and device code partitioning of complete ML applications. SODA-OPT also proposed optimization pipelines to generate pre-optimized/non-annotated code that resulted in faster custom FPGAs or ASICs designs when ingested by open-source or commercial HLS backends. Presenting results on ML workloads, both Scale-HLS and SODA-OPT focus on the generation of custom designs but don't try to target hardware templates that have been shown to be beneficial to this type of workloads [28], [22], [27]. Unfortunately, none of the above works take advantage of CGRA's reconfigurability into account for ML acceleration, which motivates ML-CGRA.

III. CGRA Architecture

This section briefly introduces our proposed CGRA architecture for the efficient acceleration of ML. Figure 1(a) shows a typical system integrating a CGRA accelerator.



(a) The integrated system (b) An adapted CGRA architecture (c) The tile architecture

Fig. 1: Example of CGRA system architecture – (a) An integrated CGRA system architecture. (b) A CGRA architecture adapted to machine learning workload. (c) The micro-architecture of a tile.

The CGRA is usually loosely coupled with a central processing unit (CPU) and can be invoked through the accelerator command interface. A direct memory access (DMA) unit handles the data communication (i.e., inputs, outputs, and control signals) between the CGRA's scratchpad memory (i.e., input/output buffers and control memory) and the host. Each CGRA tile contains a set of functional units, registers, a 10×11 crossbar, and a control memory. To enable efficient acceleration of ML workloads, we augment this canonical CGRA template with additional hardware components as highlighted in Figure 1(b). Specifically, instead of the conventional mesh on-chip network, the CGRA tiles are inter-connected in a king mesh (i.e., 8 directions) topology to facilitate the routing of the dataflow (as shown in Figure 4). In addition, the input buffers can stall for a configurable number of cycles before feeding data and support double-buffering. Moreover, the design includes dedicated MAC (multiplication followed by addition) and MAX (compare two inputs and return the larger one) functional units to enable single-cycle execution of common ML operations.

The conventional approach represents the outer loop of a kernel as a dataflow graph (DFG) and maps it onto the CGRA using a heuristic modulo scheduling algorithm [16]. The algorithm tries to optimize the Initiation Interval (II) considering the resource minimum II (ResMII) and the recurrence minimum II (RecMII) as constraints. For example, the loop shown in Figure 2(c) can be mapped onto a 3×3 CGRA with an II of 4 (i.e., a new iteration is started every 4 cycles). The II is determined by the recurrence dependency (i.e., n0-¿n4-¿n6-¿n8-¿n0 in Figure 2(d)). In this way, the entire loop executes $(4\times N+2)$ cycles (i.e., 3602 cycles) where N indicates the loop boundary and 2 is the epilogue.

The approach works at the instruction level, i.e., at the LLVM intermediate representation (IR). However, such a low level might be unnecessarily complicated with additional performance overhead for ML workloads where the loop boundary is determined by the tensor size and the data access is in sequential. In other words, the conventional per kernel mapping strategy lacks the higher-level knowledge of the ML operations. On the other hand, CGRAs can be actually configured as a form of spatial accelerator that is optimal for a specific ML operation. Figure 3 a) and b) show how to configure the CGRA as a systolic array for matrix multiplication and as an optimal spatial accelerator for the fused element-wise operations, respectively.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 09:42:47 UTC from IEEE Xplore. Restrictions apply.

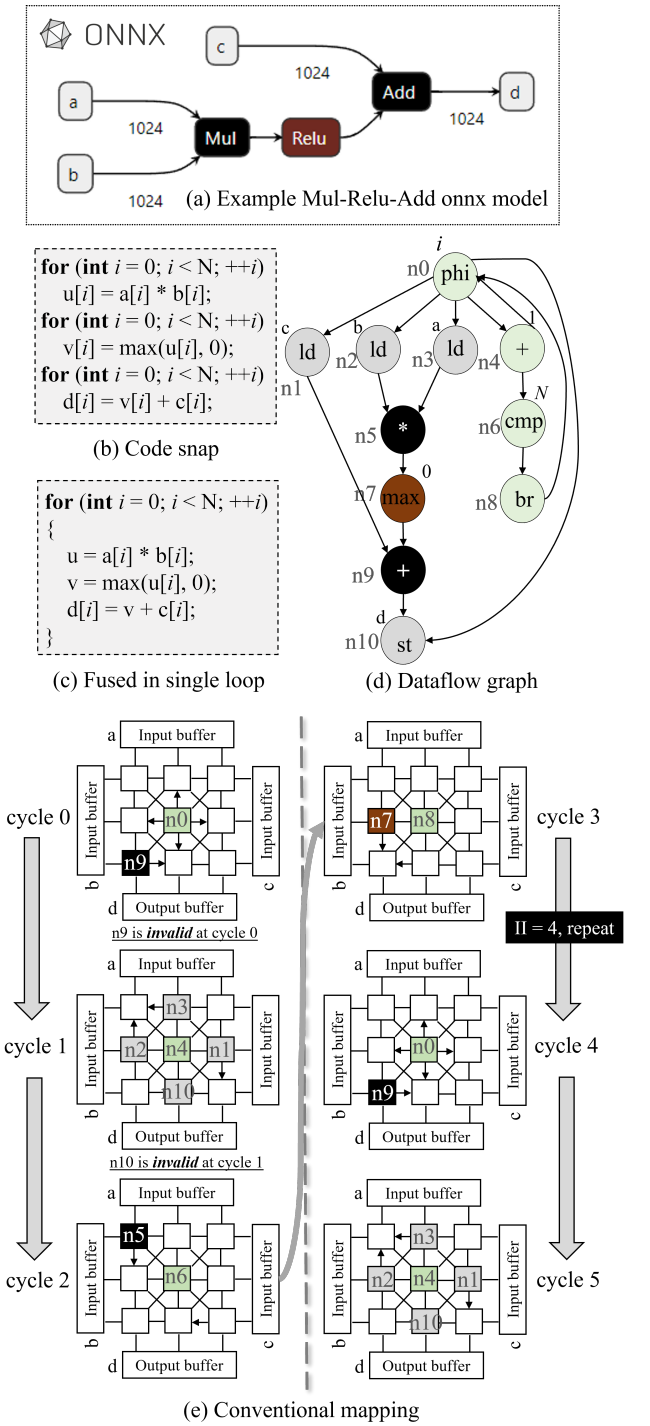


Fig. 2: Model mapping on CGRA – Typically, the CGRA mapper takes a single-loop kernel at a time, represents it as a dataflow graph, and maps it trying to achieve the lowest Initiation Interval.

We use a walk-through example (Figure 4) to show how the pre-configured CGRA achieves better acceleration speedup than the conventional loop kernel (fused element-wise operations) mapping. The kernel takes three tensors as inputs and has one output tensor as shown in Figure 2(a)-(c). The inputs `a`, `b`, and `c` are located in the input buffers while the output buffer holds the output `d`. The inputs `b` and `c` flow across the CGRA tiles in the diagonal direction while the input `a` flows

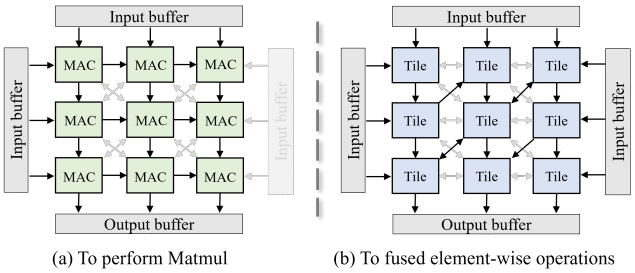


Fig. 3: Pre-configuring the CGRA as a optimal spatial accelerator for a specific ML operation – (a) Configuring the CGRA as a systolic array for `matmul`. (b) Optimized configuration for accelerating fused element-wise operations.

vertically after stalling for 2 cycles to wait for input `b`'s arrival. We can see that it takes $(N \div dim + 2 \times (dim - 1))$ cycles (i.e., 304 cycles) in total to complete the entire computation on the three input tensors. This provides an 11.8× speedup compared with the conventional mapping*. Therefore, recognizing the computational patterns/operations at a higher level to accelerate them with the pre-configured CGRA templates without going through the conventional lower-level DFG mapping can significantly improve the overall performance.

IV. ML-CGRA Framework

ML-CGRA is an integrated framework built on top of MLIR (see Figure 5) that compiles machine learning models described in popular open-source ML frameworks, such as Pytorch, ONNX, and Tensorflow, to low-level codes for CGRAs. To achieve globally automatic compilation, we implement CGRA-Opt and CGRA-Runner, the key components of ML-CGRA, which bridge the programmability and mapping gap between the ML models and CGRAs targets. CGRA-Opt contains a set of lowering and optimization passes to enable the efficient mapping of ML models on CGRAs that achieve superior speedup. Conventional CGRA compilation focuses on per-kernel mapping by lowering it into LLVM IR targeting minimized II, which loses the higher-level information of ML operations. In contrast, ML-CGRA keeps such information and enables, when possible, direct offloading of the ML operations at a higher level of abstraction. In addition, ML models always include computational patterns that can be identified by the compiler and mapped onto pre-configured and optimized accelerator templates. Finally, to help us develop accelerators and unbiasedly evaluate the framework effectiveness, we introduce CGRA-Runner, which provides the runtime library and the interfaces to facilitate cycle-level and RTL simulation. In the following sections, we introduce the implementation details of the proposed framework.

A. CGRA-Opt

We created a `cgra-dialect` and transformation passes to build CGRA-Opt. The transformation passes include three important optimizations (i.e., Operation Fusion, Loop Tiling, and Pattern Matching) and Offloading.

*Although conventional mapping could be better by leveraging other optimization techniques such as operation fusion and loop unrolling, it requires DFG transformations, design space exploration, and non-trivial hardware support, which are not the focus of this work

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 09:42:47 UTC from IEEE Xplore. Restrictions apply.

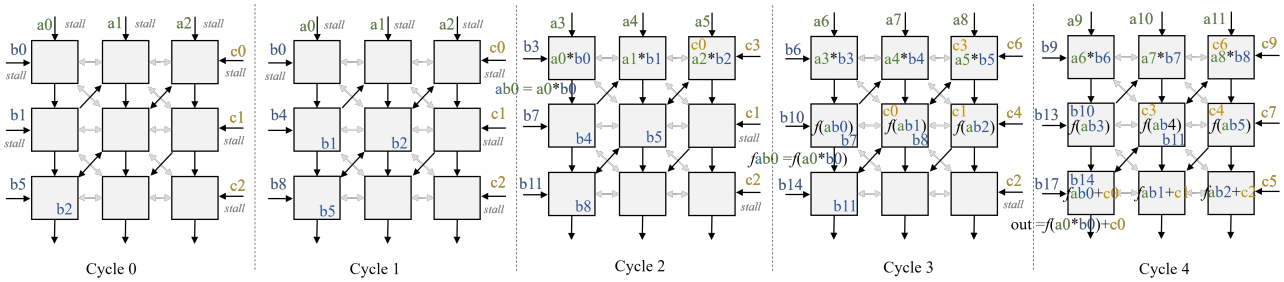


Fig. 4: Element-Wise Operations Fusion and the Corresponding Data-Flow Acceleration on CGRA – The inputs `b` and `c` flow across the CGRA tiles in diagonal direction while the input `a` flows vertically.

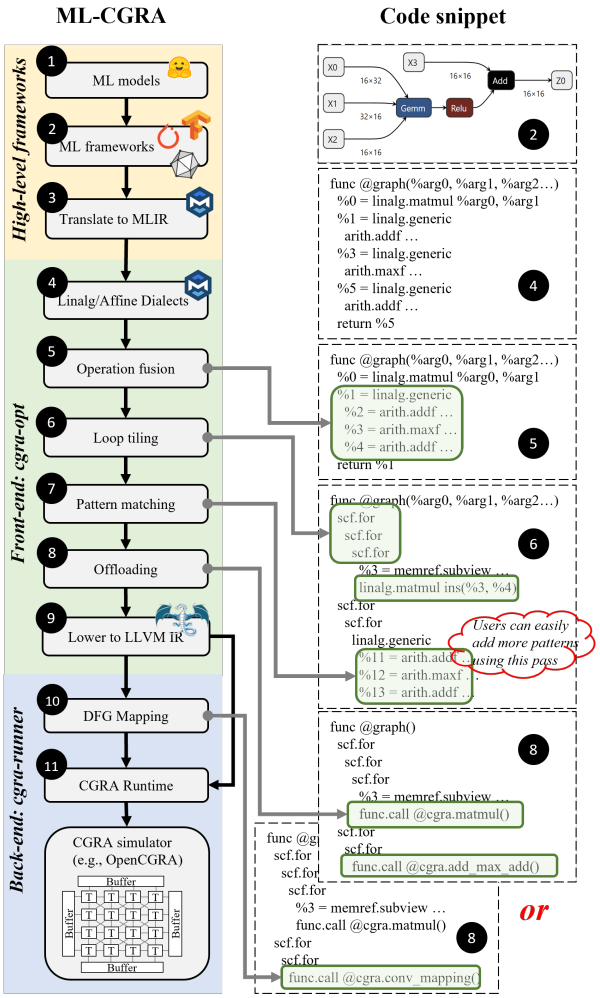


Fig. 5: Overview of ML-CGRA Framework with the corresponding code snippets after certain stages – The fused operations (at stage 8) can be lowered into the function call of `cgra.add_max_add()` or `cgra.conv_mapping()` depending on whether a pre-configured CGRA is provided by the user. Note that `cgra.conv_mapping()` here stands for conventional mapping on CGRA.

Operation Fusion – In traditional loosely coupled accelerators, including CGRAs, data movement is time-consuming and power-hungry. This is an overreaching issue. To overcome this problem and accelerate computation ML-CGRA leverages the `--linalg-fuse- elementwise-ops` pass to fuse element-wise operations on tensors. This improves the utilization of the compute resources of CGRA and minimizes the data movement between the fusible elementwise operations. As an example, Figure 5 stage 5 presents a detailed fusion for three operations including `addf, maxf, and addf,` during which the fusion number can be self-modified.

Loop Tiling – As one of the most precious resources in hardware platforms, on-chip memory/registers are limited in CGRAs, similar to other hardware. To maximize their utilization, we implement a loop tiling pass to tile the input/output tensors based on the size of the input/output buffers. Besides, to further improve framework efficiency, we apply the double-buffering technique on ML-CGRA, as an option, which can minimize the delay in data communication.

Pattern Matching – As shown in Figure 2 and Figure 4, mapping a tensor-based ML operation to the CGRA in a specific way can often outperform the conventional mapping. Thus, we implement a pattern-matching pass to recognize the patterns/operations that the pre-configured CGRA templates can directly accelerate without lowering into LLVM-IR. Note that users can easily add any patterns by using this pass. To be specific, the CGRA can be configured as shown in Figure 3 to perform efficient acceleration on the fused pattern `mul-relu-add`. Then, all the `mul-relu-add` operations in the given ML model can be recognized by pattern matching pass, which avoids lowering them into the LLVM-IR to perform the conventional kernel mapping as shown in Figure 2(e). Similarly, `matmul` operations can be recognized by the pattern matching pass to be efficiently accelerated by configuring the CGRA as a systolic array implementing an efficient matrix multiplication. Note that not all operations/patterns have to be accelerated with a pre-configured CGRA mapping. ML evolves often, and new patterns from an upcoming ML model can always be lowered into LLVM-IR to perform the conventional mapping for CGRA acceleration. As shown in the last code snippet of Figure 5, if the user does not allow the fused operations to be pattern matched, it will be lowered as a function call of `cgra.conv_mapping`, which invokes the conventional DFG mapping.

Offloading – Once the above passes are complete, an offloading pass will lower each operation into a CGRA function call (i.e., Accelerator Cmd as shown in Figure 5 stage 8). The CGRA runner can also use the CGRA function call to evaluate their execution performance.

B. CGRA-Runner

To support the full end-to-end design flow for specializing and implementing CGRAs, researchers proposed several CGRA-based simulators (e.g. OpenCGRA [19] and CGRA-ME [6]). These simulators model a CGRA at different levels of

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 09:42:47 UTC from IEEE Xplore. Restrictions apply.

Model Name	Domain	# Attention Heads	# Layers	Parameters
MiniLM [23]	Text Classification	12	6	66M
Sentence-BERT [17]	Sentence Similarity	12	12	120M
CamemBERT [14]	Token Classification	12	12	110M
Vision Transformer [8]	Image Classification	12	12	86M

TABLE I: Detailed information of the target machine learning models – the models are from computer vision and natural language domains.

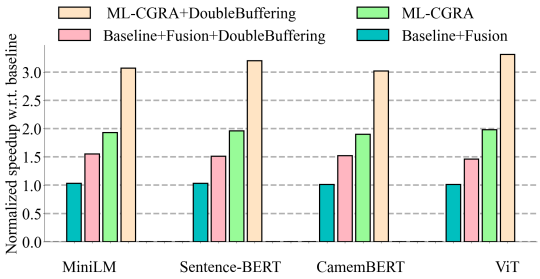


Fig. 6: Normalized acceleration speedup of different optimizations across models with respect to the baseline – The baseline leverages the conventional kernel mapping for models on a 4×4 CGRA.

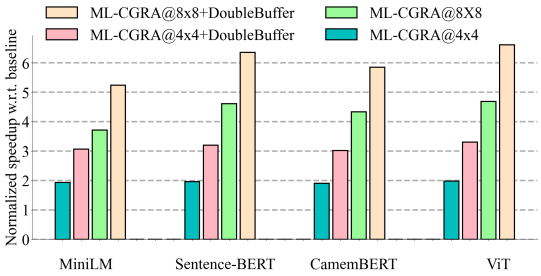


Fig. 7: Normalized speedup for models running on different CGRAs – The baseline leverages the conventional kernel mapping for models on a 4×4 CGRA. Each input/output buffer is 1KB and 4KB in the 4×4 and 8×8 CGRAs, respectively.

abstraction, provide compiler support, verification at different granularities, simulation, generation of synthesizable Verilog, and characterization. Thanks to those simulators and their interfaces, we can transform our CGRA function calls generated by the offloading stage to the simulator's runtime library, allowing us to achieve cycle-level and RTL simulation and obtain the exact execution time for performance evaluation.

V. Evaluation

To illustrate the potential of ML-CGRA, we evaluate a set of state-of-the-art machine learning models from diverse domains accelerated by CGRAs. We compare the performance (i.e., execution time running on the same CGRA) of the code generated by ML-CGRA against the conventional DFG mapping methodology. We also discuss how the proposed framework adapts to CGRAs of different dimensions, to ensure its applicability to larger devices.

A. ML Models

Computer vision and natural language processing are key applications of ML models. A key component of these models is the attention mechanism. The computational patterns and flow of data across computations of attention mechanisms perfectly match the features provided by a CGRA. Table I presents detailed information about the attention-based models used in our evaluation.

B. Experimental Results

Given an ML model provided by hugging face, we can compile it to the `Linalg` MLIR dialect through Torch-MLIR. The `Linalg` representation of the model serves as the input to ML-CGRA. ML-CGRA can apply several optimization passes to generate the optimal code for a target CGRA configuration. We implement a cycle-accurate CGRA simulator to obtain the execution times in terms of cycles. Note that the simulator exploits the CGRA runtime to implement the CGRA-Runner.

Baseline – We consider the conventional low-level mapping methodology of ML models on a 4×4 CGRA as the baseline. We compute data communication overheads considering a typical direct memory access (DMA) controller running at 200 MHz. We employ input buffers of 1 KB for 4×4 CGRAs and of 4 KB for 8×8 CGRAs.

Acceleration Speedup – Figure 6 shows the normalized speedups of ML-CGRAs over baseline implementation across 4 different models. To further break down the benefits from different optimizations applied in ML-CGRA, we compare baseline with 4 implementations including (1) Baseline + Fusion, (2) Baseline + Fusion & Double Buffering, (3) ML-CGRAs equipped with all optimizations (including Fusion and pre-configured CGRAs) but double buffering, and (4) ML-CGRAs, i.e. implementation (3), with double buffering. The results demonstrate that the full-version ML-CGRA (implementation (4)) is able to fuse operations automatically and accelerates specific kernel with predefined optimized patterns to obtain superior performance, leading to 3.15× over the baseline and better realizing the potentials of CGRA architecture in ML tasks. Furthermore, the results also show that the double buffering technique brings comparable speedups for both baseline (1.5× from implementation (1) to (2)) and ML-CGRA (1.65× from implementation (3) to (4)). For implementation (1) where the baseline is augmented with operation fusion, although operation fusion significantly reduces the data movement, most of the models are still bounded by the computation loads of matrix multiplications, which leads to on average only 3% improvement in terms of speedup across all the models.

Scalability – We also evaluate the scalability of ML-CGRA by deploying the models onto different sizes of CGRAs (e.g. 4×4 and 8×8). As figure 7 shows, ML-CGRA framework can effectively accelerate machine learning algorithms on 8×8 CGRAs and obtain about 90.17 % (123.61% with double-buffering enabled) improvement on speedup compared with 4×4 CGRAs, which is reasonable as the system quickly changes to communication-bound from compute-bound when the hardware resources increase by 4 times. Existing tiling pass in ML-CGRA targets output-stationary. However, ML-CGRA is built on top of MLIR, which makes it easier to plug in with open-source tiling/data-movement passes to maximize the data reuse and potentially improve the scalability further.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 09:42:47 UTC from IEEE Xplore. Restrictions apply.

VI. Conclusion

This paper proposes ML-CGRA, an open-source integrated compilation framework to enable efficient machine learning implementation and acceleration on CGRAs. Under ML-CGRA, users can effectively compile machine learning models from high-level frameworks to optimized low-level codes for CGRAs. Compared with a conventional CGRA implementation for machine learning kernels, ML-CGRA provides an end-to-end solution to specifically accelerate machine learning models with pre-configured CGRAs and to improve on average the execution performance with 3.15× and 6.02× speedup on 4×4 and 8×8 CGRAs for a diverse set of models, respectively.

Acknowledgments

The authors acknowledge the support from the University of Rochester. This research was partially supported by the U.S. DOE Office of Science, Office of Advanced Scientific Computing Research, under award 66150: "CENATE - Center for Advanced Architecture Evaluation". This research was also supported by the Software Defined Accelerators for Utra-Low Latency Reasoning (SODA-ULTRA) project in the AT SCALE Initiative and the Software Defined Architectures for Portability and Performance (SODAPOP) project in the DMC Initiative under the PNNL's Laboratory Directed Research and Development (LDRD) program.

References

[1] N. B. Agostini, S. Curzel, V. Amatya, C. Tan, M. Minutoli, V. G. Castellana, J. Manzano, D. Kaeli, and A. Tumeo. An MLIR-based Compiler Flow for System-Level Design and Hardware Acceleration. In *IEEE/ACM International Conference on Computer-Aided Design*, ICCAD'22, San Diego, CA, 2022. IEEE.

[2] S. Ahmad, S. Subramanian, V. Boppana, S. Lakka, F. Ho, T. Knopp, J. Noguera, G. Singh, and R. Wittig. Xilinx First 7nm Device: Versal AI Core (VC1902). In *2019 IEEE Hot Chips 31 Symposium (HCS)*, pages 1–28, Los Alamitos, CA, USA, aug 2019. IEEE Computer Society.

[3] K. Ando, S. Takamaeda-Yamazaki, M. Ikebe, T. Asai, and M. Motomura. A multithreaded CGRA for convolutional neural network processing. *Circuits and Systems*, 8(6):149–170, 2017.

[4] I. Bae, B. Harris, H. Min, and B. Egger. Auto-tuning CNNs for coarse-grained reconfigurable array-based accelerators. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 37(11):2301–2310, 2018.

[5] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 578–594, Carlsbad, CA, Oct. 2018. USENIX Association.

[6] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, and J. Anderson. CGRA-ME: A unified framework for CGRA modelling and exploration. In *2017 IEEE 28th international conference on application-specific systems, architectures and processors (ASAP)*, pages 184–189. IEEE, 2017.

[7] W. C. Developers. Wave Computing. https://wavecomp.ai.

[8] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*, 2021.

[9] M. Emani, V. Vishwanath, C. Adams, M. E. Papka, R. Stevens, L. Florescu, S. Jairath, W. Liu, T. Nama, and A. Sujeeth. Accelerating Scientific Applications With SambaNova Reconfigurable Dataflow Architecture. *Computing in Science & Engineering*, 23(2):114–119, 2021.

[10] X. Fan, D. Wu, W. Cao, W. Luk, and L. Wang. Stream processing dual-track CGRA for object inference. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 26(6):1098–1111, 2018.

[11] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation. In *CGO*, pages 2–14, Seoul, South Korea, 2021. IEEE.

[12] J. Lee and J. Lee. NP-CGRA: Extending CGRAs for Efficient Processing of Light-weight Deep Neural Networks. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1408–1413. IEEE, 2021.

[13] M. Liang, M. Chen, Z. Wang, and J. Sun. A CGRA based neural network inference engine for deep reinforcement learning. In *2018 IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, pages 540–543. IEEE, 2018.

[14] L. Martin, B. Müller, P. J. O. Suárez, Y. Dupont, L. Romary, É. V. de la Clergerie, D. Seddah, and B. Sagot. CamemBERT: a Tasty French Language Model. *CoRR*, abs/1911.03894, 2019.

[15] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer. Timeloop: A Systematic Approach to DNN Accelerator Evaluation. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 304–315, 2019.

[16] B. R. Rau. Iterative modulo scheduling: An algorithm for software pipelining loops. In *MICRO'94*, 1994.

[17] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *CoRR*, abs/1908.10084, 2019.

[18] C. Tan, N. B. Agostini, T. Geng, C. Xie, J. Li, A. Li, K. J. Barker, and A. Tumeo. DRIPS: Dynamic Rebalancing of Pipelined Streaming Applications on CGRAs. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 304–316. IEEE, 2022.

[19] C. Tan, C. Xie, A. Li, K. J. Barker, and A. Tumeo. OpenCGRA: An open-source unified framework for modeling, testing, and evaluating CGRAs. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*, pages 381–388. IEEE, 2020.

[20] C. Tan, C. Xie, A. Li, K. J. Barker, and A. Tumeo. Aurora: Automated refinement of coarse-grained reconfigurable accelerators. In *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1388–1393. IEEE, 2021.

[21] M. Tanomoto, S. Takamaeda-Yamazaki, J. Yao, and Y. Nakashima. A cgra-based approach for accelerating convolutional neural networks. In *2015 IEEE 9th International Symposium on Embedded Multicore/Many-core Systems-on-Chip*, pages 73–80. IEEE, 2015.

[22] TVM Developers. VTA: Deep learning accelerator stack, 2020.

[23] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *CoRR*, abs/2002.10957, 2020.

[24] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, and T. Nowatzki. Dsagen: Synthesizing programmable spatial accelerators. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 268–281. IEEE, 2020.

[25] Q. Xiao, S. Zheng, B. Wu, P. Xu, X. Qian, and Y. Liang. HASCO: Towards Agile hardware and software co-Design for Tensor Computation. In *Proceedings of the 48th Annual International Symposium on Computer Architecture*, ISCA '21, page 1055–1068. IEEE Press, 2021.

[26] H. Ye, C. Hao, J. Cheng, H. Jeong, J. Huang, S. Neuendorffer, and D. Chen. ScaleHLS: A New Scalable High-Level Synthesis Framework on Multi-Level Intermediate Representation. In *IEEE International Symposium on High-Performance Computer Architecture*, HPCA'22, pages 741–755, Seoul, South Korea, 2022. IEEE.

[27] H. Ye, X. Zhang, Z. Huang, G. Chen, and D. Chen. HybridDNN: A Framework for High-Performance Hybrid DNN Accelerator Design and Implementation. In *57th ACM/IEEE Design Automation Conference*, DAC'20, pages 1–6, San Francisco, CA, USA, 2020. IEEE.

[28] X. Zhang, H. Ye, J. Wang, Y. Lin, J. Xiong, W.-M. Hwu, and D. Chen. DNNExplorer: A Framework for Modeling and Exploring a Novel Paradigm of FPGA-based DNN Accelerator. In *IEEE/ACM International Conference on Computer-Aided Design*, ICCAD'20, pages 1–9, San Diego, CA, USA, 2020. IEEE.

[29] L. Zulberti, M. Monopoli, P. Nannipieri, and L. Fanucci. Architectural Implications for Inference of Graph Neural Networks on CGRA-based Accelerators. In *2022 17th Conference on Ph. D Research in Microelectronics and Electronics (PRIME)*, pages 373–376. IEEE, 2022.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on November 20,2025 at 09:42:47 UTC from IEEE Xplore. Restrictions apply.